



Sprint 2 Stories and Learning Plan

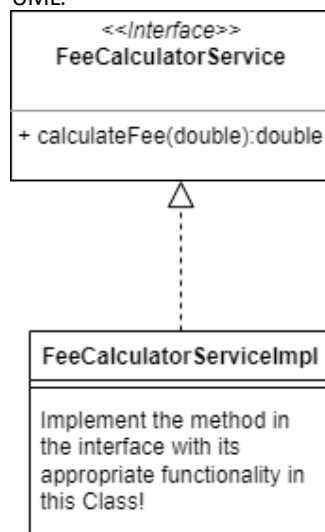
Consultant Version

Epic 2: Account management and fee calculations

Stories:

1. As a bank I want to calculate fees on accounts based on the maintained balance in order to charge the fees from the applicable accounts.
 - a. I have a FeeCalculationService interface which declares a calculateFee() method. You are expected to implement the interface using the TTD methodology of software development. You will be expected to write your failing tests and write enough code for them to pass, then refactor and generalize.
 - b. Requirement:
 - i. In 'service' package create a class named "FeeCalculatorServiceImpl". The class should implement the "FeeCalculatorService" interface.
 - ii. To be successful, tests should be implemented to assert the behaviour of the "calculateFee" method from the "FeeCalculationService" interface.
 - iii. The "calculateFee" method returns a fee value based on the account balance passed in. The variable datatype for fee and balance is double. And, the method follows a fee structure determined by the following account balance ranges:
 1. A \$20.00 fee for a balance less than or equal to \$100.
 2. A \$15.00 fee for a balance greater than \$100 but less than or equal to \$500.
 3. A \$10.00 fee for a balance greater than \$500 but less than or equal to \$1000.
 4. A \$5.00 fee for a balance greater than \$1000 but less than or equal to \$2000.
 5. A \$0.00 fee for a balance greater than \$2000.

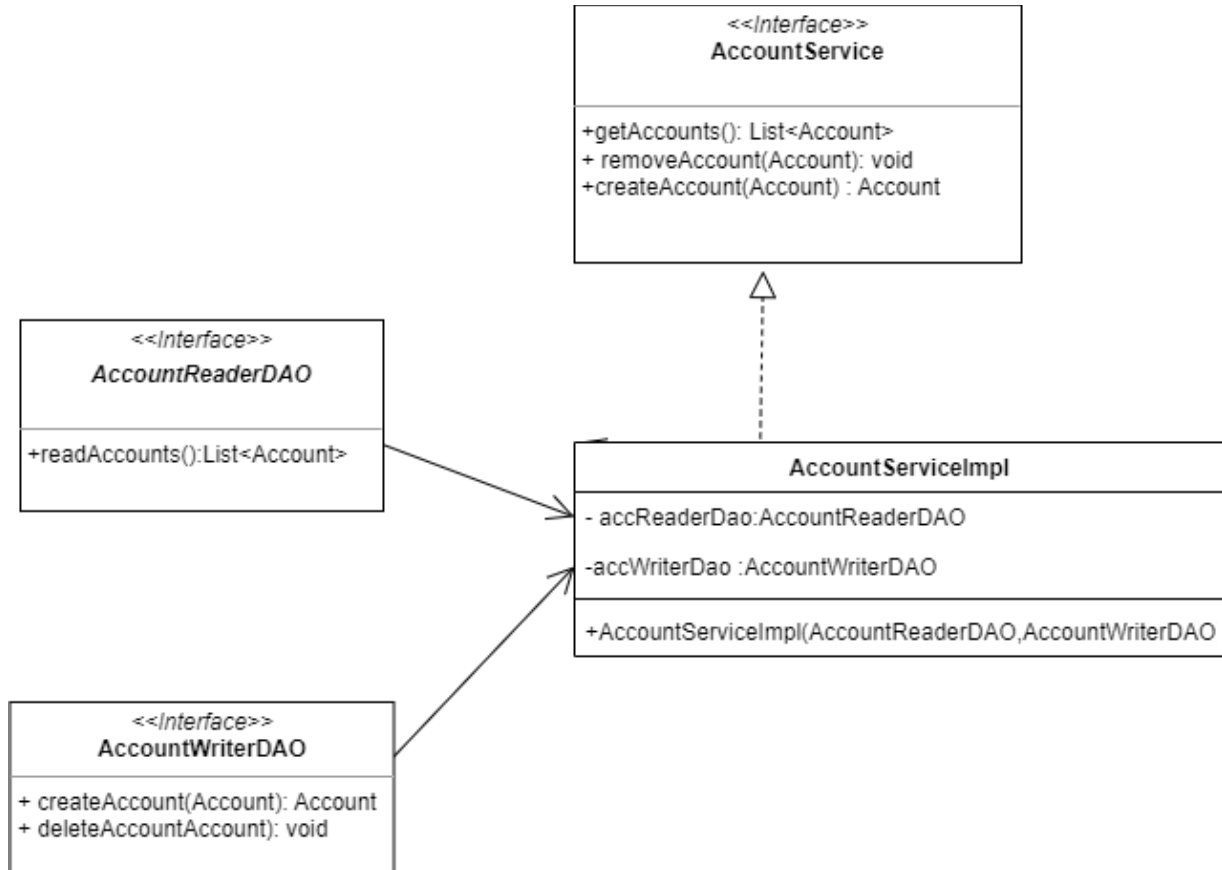
c. UML:



2. As a Teller I should be able to get a list of accounts, delete an account and ability to create a new account to perform day to day banking.
 - a. I have a AccountService interface which declares methods like createAccount(),removeAccount() and getAccounts() methods. You are expected to implement the interface using the TTD methodology of software development. You will be expected to write your failing tests and write enough code for them to pass, then refactor and generalize.
 - b. Requirement:
 - i. In 'service' package a class "AccountServiceImpl" that implements the "AccountService" interface.
 - ii. Verify that the AccountServiceImpl's getAccounts() method returns a list of accounts it receives from a call to the AccountReaderDAO's readAccounts() method.

- iii. The `removeAccount()` method takes in as an argument an account object to be removed. Verify that this method calls the `deleteAccount()` method of the `AccountWriterDAO` class passing the same account object which it received as its argument.
- iv. The `createAccount()` method takes in as an argument an account object to be created. Verify that this method calls the `createAccount()` method of the `AccountWriterDAO` class passing the same account object which it received as its argument.

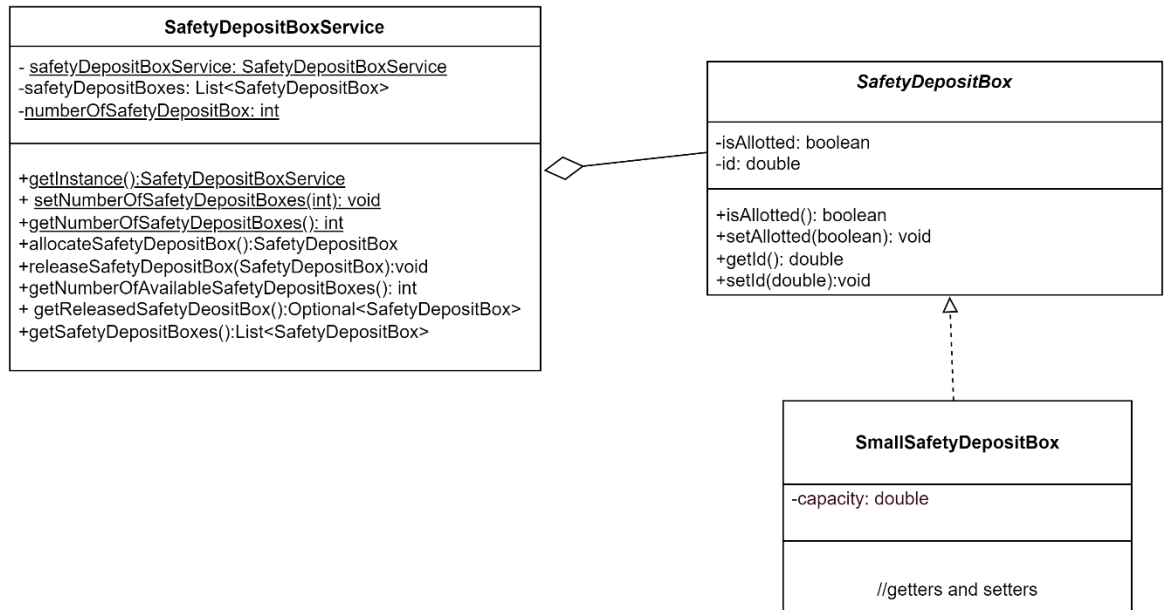
c. UML:



3. As a bank we need our application to allocate and release safety deposit boxes in a bank's branch so customers can be allocated safety deposit boxes if available and can release them if they no longer need it.
 - a. Note: Please create all the tests required to pass the criteria outlined. Along with the code necessarily to pass the tests. The final code should follow the UML provided.
 - b. Requirements:
 - i. Create a `SafetyDepositBoxService` class used to request and release safety deposit boxes in a bank. The service has a pool of Safety Deposit Boxes. These Safety Deposit Boxes are allocated and released per the user's requirements.
 - ii. Upon request, the program checks if a Safety Deposit Box is available. If a box is available, it should be allocated to the client. If a box is unavailable, and the number of total boxes is less than the Maximum limit, then the service must create a new Safety Deposit Box instance and allocate it. If there is no available box and the maximum limit of boxes has been reached, the user must be informed that no new boxes can be create and no existing boxes are available. Also, if requested, the service should be able to show how many Safety Deposit Boxes are available. Hint: Make use of the Object Pool Design Pattern to implement this model.
 - c. `SafetyDepositBoxService` description:
 - i. Create the class to be a Singleton. As a result, the only way to get an instance of this class should be through a `getInstance()` method.

- ii. `allocateSafetyDepositBox()` method when invoked is expected to produce the following outcomes:
 - 1. If a safety deposit box is currently available in the pool, the method should return it. You can call the `"getReleasedSafetyDepositBox()"` for this.
 - 2. If no box is available, in the pool:
 - a. The method should check if the maximum limit has been reached. If the limit has not been reached, a new Safety Deposit Box should be created and returned.
 - b. Otherwise, if the maximum limit has been reached the method should wait until a safety deposit box becomes available. During the waiting period, you can use a flag to indicate that the thread is waiting (for testing purposes). Once a box becomes available, the method should return the released safety deposit box to the client.
 - iii. `getNumberOfAvailableSafetyBoxes()` method should return the number of safety boxes available for allocation
 - iv. `getReleasedSafetyDepositBox()` method should iterate through the list of safety deposit boxes and return the first available safety deposit box
 - v. `releaseSafetyDepositBox()` method takes a `Safetydepositbox` object as an input argument and makes the box available by returning the object to the pool for future use. Additionally, the method should notify any waiting threads that a box has been released and is now available.
 - vi. To ensure thread safety, all methods that interact with shared resources, specifically the safety deposit boxes, should be synchronized.
- d. Testing Requirements:
- i. Make sure to create test cases that cover all the functionality of the `SafetyDepositBoxService` class.
 - ii. **Optionally: The Safety deposit Box Object pool must be shrinkable. This means that each time a safety deposit box is released it must be deleted.**
 - iii. Set the `numberOfSafetyDepositBoxes` to two
 - iv. Write a minimum of two test case methods. Name the test case methods using the appropriate naming conventions for best practices
 - 1. Test1:
 - a. The test case method creates two threads. Each thread requests a safety deposit box. And holds the box for 5 seconds before releasing it.
 - b. This method should verify that no thread was kept waiting.
 - 2. Test2:
 - a. The test case method creates three threads. Each thread requests a safety deposit box. And holds the box for 5 seconds before releasing it.
 - b. This method should verify that a thread was kept waiting for a safety deposit box.

e. UML:



Spike- As a developer I want to enhance my Java development skills to be able to write more scalable, reliable and readable code.

Sprint Goal: Enhance my Java Development skills to be able to write more scalable, reliable, secure and readable code.

Day	9:00 AM	11:15 AM	2:00 PM	3:45 PM	5:00 PM
1	Sprint Planning	Exceptions	JUnit Coach session	JUnit practice	Problem solving with coach
2	JUnit Coach Review followed by Mockito	Mockito Practice	Mockito Practice	JUnit and Mockito advanced Demo	Problem solving with coach
3	Collections	Collections	Generics	Generics	Problem solving with coach
4	Sonar Lint	Run Sonar Lint on previous story	Comparisons	Logging	Problem solving with coach
5	Work on stories	Work on stories	Work on stories	Work on stories	Problem solving with coach
6	SOLID	Class discussions	Design patterns	DP Task	Problem solving with coach
7	Threading	Threading	Threading	Threading	Problem solving

				with coach
8	Optionals	Work on stories	Work on stories	Work on stories
9	Work on stories	Work on stories	Work on stories	Work on stories
10	Sprint Review	Sprint Review	Complete all pending exercises and submit	Complete all pending exercises and submit

Acceptance Criteria:

- Able to follow TDD and write unit tests using Junit and Mockito
- Ability to choose the right collection from the Java Collections Api to solve problems.
- Able to write generic programs.
- Can scan my code using sonar lint and write clean code
- Understand the usage of comparisons and can efficiently use logging in my code
- Can adhere to SOLID principles
- Can use and appreciate the use of design principles to solve problems
- Can write programs to work in multi threaded environment without fail
- Can understand concepts like optionals, streams and lambdas
- Submit all assigned exercises to coach using Git

Assessment Rubrics:

- **Task Completion in time including learning.**
- **Participation in group discussion**
- **Interviewing skills**
 - **Problem Solving and Algorithmic Thinking**
 - **Communication and Technical ability**
- **Project**
 - Refactor successfully done, matching the UML.
 - Clean Code, Coding Conventions, Design Standards, Optimization Successfully implemented
 - Story 1 requirements
 - Story 2 requirements
 - Story 3

- Singleton class implemented correctly
- allocateSafetyDepositBox() implemented correctly
- getReleasedSafetyDepositBox() implemented correctly
- releaseSafetyDepositBox() implemented correctly
- getNumberOfAvailableSafetyDepositBoxes() implemented correctly
- Testing Requirements: Unit Tests for SafetyDepositBoxService class made correctly
- Implemented Optional task correctly
- Test case1
- Test case2